

CLICK PLC Two-Tank Automated Batch-Mixing Station

PLC-Authoritative Sequencing with Ignition Perspective SCADA

Six single-responsibility subroutines, a next-state-commit sequencer, a twelve-alarm condition / delay / trigger / lock-in / alert block, a closed-loop PID temperature controller acting on a first-order thermal model, a validated recipe-download handshake, and eight physical outputs each written from exactly one dedicated image bit.

Training scope - not a chemical or food-grade process

This document describes a bench-top PLC and SCADA demonstration built entirely on a CLICK trainer using switches, lamps, timers, and simulated analog math. It is **not** a specification for an actual batch-mixing skid. No real tanks, valves, agitators, or heaters are involved; levels and temperature are simulated inside the PLC. A real process would additionally require certified level and temperature instrumentation, pressure relief, chemical compatibility review, hazardous-area classification where applicable, and independently supervised safety interlocks.

Four design rules used throughout

1. The PLC owns the sequence. Ignition sends requests, setpoints, and recipe data; it never drives the state machine directly. **2. Requests are momentary, state is persistent.** Every HMI and pushbutton action becomes a one-scan trigger, then a persistent command latch that the sequencer clears once it has acted. **3. Physical outputs have one writer.** C301--C308 are the only logical commands for the eight lamps; each has exactly one OUT coil in the subroutine that owns its logic, and the I/O subroutine copies each one to its Y address with a plain contact-to-coil rung – never a branch, never a combination. **4. Every alarm is a lock-in, not a level.** A condition is delayed, edge-detected into a one-scan trigger, sealed into an alarm bit that only an explicit reset can break, and mirrored into a silenceable alert – the same four-stage pattern used in the Duty/Standby Pump Lead-Lag project.

What changed in this revision

This revision closes the gaps in the first draft. Read it before entering the program if you have the earlier version open.

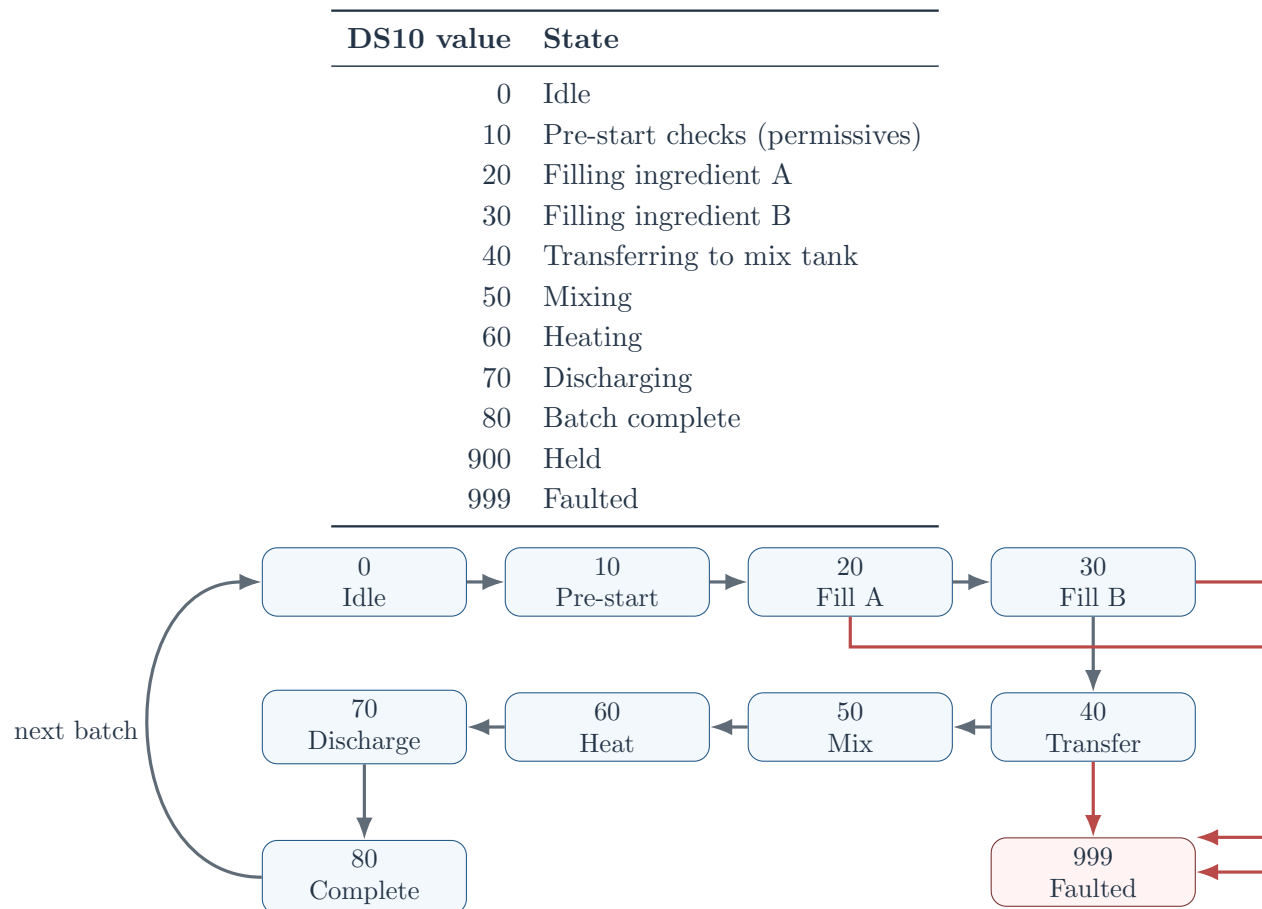
- Alarms are implemented**, not merely tabulated. Twelve alarms now live in their own subroutine using the lead-lag condition → delay → trigger → sealed alarm → silenceable alert pattern, with a first-out code register and an explicit reset rule.
- C307 and C308 are now real bits.** All eight equipment/indicator image bits map 1:1 to Y001--Y008; the running lamp and alarm horn are no longer combination logic written at the output.
- PID and tank simulation moved to their own subroutine**, separate from sequencing.
- Command-latch address collision fixed.** The first draft wrote C55 as the latch for Start, Hold, Resume *and* Abort simultaneously.
- Timer double-booking fixed.** T3 was used as both the transfer and discharge step timeout; discharge now uses T6.
- Same-scan state cascade fixed** with a next-state register (DS13) committed once per scan – see 13.
- Clock bits corrected.** SC5 and SC7 are square waves that toggle every 50 ms and 500 ms, not pulses. They

are now taken through **One Shot** coils.

8. **PID register offsets made self-consistent** against the CLICK loop table, and the autotune hysteresis range corrected to 0.1–10 % of PV span.
9. **Recipe download handshake implemented** with staging registers, so a partially entered recipe can never reach the active recipe block.
10. **Silence added** alongside Reset, so the horn can be quieted without unlocking a fault.
11. **Edge detection corrected to match the CLICK instruction set.** The CLICK has no rising-edge contact – one-shots are produced by ticking **One Shot** on an OUT coil. All edge generation now lives in three named blocks, headed by the new **SBR-OS** subroutine, which derives a one-scan partner for every request bit at +900. Request bits themselves arrive from Ignition and the panel as raw levels and are never rewritten by the PLC.

1 What the Batch Sequence Does

The station simulates a two-ingredient batch: Tank A and Tank B each fill to a recipe-defined target, both ingredients transfer into a mixing tank, the mixer runs for a recipe-defined time, the batch heats to a setpoint under closed-loop PID control, and the finished batch discharges. The sequence is tracked with a single integer state register so that Ignition, an HMI, or a technician with a handheld programmer can always tell what the process is doing by reading one register.



Held (900) is reachable from any active step through a hold request and returns to the same step on resume; it is omitted above for clarity, as is the timeout path from Mix, Heat, and Discharge into Faulted. Any sealed alarm forces 999 from any active step.

Why a numbered state register instead of separate step-active bits

A single integer that both the PLC and Ignition read makes online troubleshooting and HMI status text trivial: one comparison shows the active step, one historized tag trends the whole sequence, and a technician with only a handheld programmer can diagnose the process without decoding a dozen coincident bits.

2 Operating Modes

Off – outputs de-energized, sequence forced to Idle.

Manual – individual equipment commands from Ignition pass through permissives and alarm interlocks but are not sequenced; used for commissioning. Manual commands enter the same C301--C306 decode rungs as the sequencer, so they cannot bypass the single-writer rule.

Automatic – the state machine runs the full batch.

Maintenance / Simulation – allows the fault-injection switch and accelerated presets for classroom demonstration; never enabled during an active batch.

X007 selects Auto (closed) or Manual (open). The PLC, not Ignition, decides whether a requested mode change is accepted, and rejects a change to Manual while DS10 is between 10 and 80 unless the batch is first held or aborted.

HMI command convention

Every HMI action is a momentary request bit (C20--C26) written by Ignition and returned to 0 after acknowledgement. The PLC replies with **StartAccepted**, **Running**, **Held**, **Faulted**, and **CommandRejected** status bits. No HMI button ever holds a physical output on directly.

3 Address Map and Signal Ownership

3.1 Physical I/O

Address	Assigned function	Address	Assigned function
X001	Start pushbutton	Y001	Valve A command lamp
X002	Stop pushbutton	Y002	Valve B command lamp
X003	Hold pushbutton	Y003	Transfer pump command lamp
X004	Resume pushbutton	Y004	Mixer command lamp
X005	Abort pushbutton	Y005	Heater command lamp
X006	Alarm reset / acknowledge pushbutton	Y006	Discharge valve command lamp
X007	Auto (closed) / Manual (open) selector	Y007	Running status lamp
X008	Simulated feedback-fault injection switch	Y008	Alarm horn / lamp

The trainer's eight inputs are fully committed, so **Silence has no pushbutton**; it exists only as the Ignition request bit C26. This is a deliberate trade rather than an omission: reset is the safety-relevant action and keeps the hard-wired button, while silence is a convenience and can live on the HMI. If a ninth input becomes available, wire it to C9 and add one rung to the trigger group.

3.2 Bit memory

Address	Purpose
C1--C8	Input image of X001--X008. Every rung outside the I/O subroutine reads these, never an X address.
C10	PERMISSIVE_MISSING (inverse of C13), for the diagnostics screen
C13	PERMISSIVES_OK
C20--C26	Ignition momentary requests: start, stop, hold, resume, abort, reset, silence
C30	Ignition recipe-download request (momentary)
C31, C32	Recipe download accepted / rejected (PLC reply, one scan)
C40--C43	Command latches: start, hold, resume, abort (persistent)
C44--C47	Command consumed acknowledgements: start, hold, resume, abort (one scan)
C48	STEP_ENTERED – one scan on any DS10 transition
C50--C56	Command triggers, one scan: start, stop, hold, resume, abort, reset, silence
C57	ANY_ALARM – OR of the twelve sealed alarms
C110	TEMP_IN_BAND
C111	HEARTBEAT_CHANGED – one scan when DS51 advances
C112	RECIPE_VALID – all four staged recipe values in range
C120	SIM_TICK – one scan per second, One Shot on SC7
C121	PID_TICK – one scan per 100 ms, One Shot on SC5
C130--C132	Simulated valve A, valve B, and pump position feedback
C210--C215	Ignition manual equipment commands (Manual mode only)
C301--C308	Equipment and indicator image bits – the single writers for Y001--Y008
C600--C611	Alarm condition bits (level, stage 1)
C620--C631	Alarm triggers (one scan, stage 3)
C640--C651	Alarm sealed alarms (lock-in, stage 4)
C660--C671	Alarm alerts (silenceable, stage 5)
C700+	PID Loop 1 configuration bits (Cn), allocated by the PID Setup dialog
C901--C906	One-shot partners of the pushbutton image bits C1--C6
C920--C926	One-shot partners of the Ignition request bits C20--C26
C930	One-shot partner of the recipe-download request C30

The four alarm rows are index-aligned

Alarm i (where $i = 1 \dots 12$) always occupies condition $C(599+i)$, trigger $C(619+i)$, sealed alarm $C(639+i)$, and alert $C(659+i)$. Fixed offsets of exactly 20 between rows mean you can read any alarm's full chain by adding 20 twice, and Ignition can browse the block as four contiguous twelve-bit groups instead of a scattered list. Adding a thirteenth alarm costs one rung in each of four adjacent groups and no renumbering.

The one-shot bit for any request is its own address plus 900

Requests arrive from Ignition and from the panel as plain *level* bits, exactly as they are written – nothing on the SCADA side is asked to produce a pulse, and no request bit is ever overwritten by the PLC. SBR-OS then derives a one-scan partner for each one at a fixed offset of 900: C1 → C901, C6 → C906, C20 → C920, C26 → C926, C30 → C930.

There is nothing to look up and nothing to allocate. If a ninth input is ever wired for Silence at C9, its one-shot is C909 without a decision being made; C907 and C908 are reserved for the mode switch and

the fault-injection switch should either ever need an edge, and are deliberately not generated today because nothing edges them. The same reasoning as the twenty-offset alarm block: a fixed arithmetic relationship costs one unused address here and there, and buys a rule that cannot be misremembered.

3.3 Data registers

Address	Assigned function	Address	Assigned function
DS10	Sequence state	DS11	Held step, captured for Resume
DS13	Next state (sequencer scratch)	DS20	Recipe: Tank A fill target, %
DS21	Recipe: Tank B fill target, %	DS22	Recipe: mix time, seconds
DS23	Recipe: temperature setpoint, °F	DS24	Recipe number
DS30	Simulated Tank A level, %	DS31	Simulated Tank B level, %
DS32	Simulated mix-tank level, %	DS33	Temperature, °F, integer mirror
DS40	Batch number	DS41	Batch duration, seconds
DS42	Batches completed (retentive)	DS43	Batches rejected (retentive)
DS50	PLC heartbeat mirror, PLC → Ignition	DS51	Ignition heartbeat counter
DS52	Heartbeat last value (internal)	DS60	First-out alarm code, 1–12
DS70--DS74	Recipe staging, Ignition-writable	DS100+	PID Loop 1 DS _n block
DF10	Ambient temperature T_{amb} , 70.0	DF11	Process gain K , 1.5 °F per % output
DF12	Time constant τ , 60.0 s	DF13	Integration timestep dt , 0.1 s
DF14	Settling band low, DS23–2.0	DF15	Settling band high, DS23+2.0
DF16	Overtemperature limit, DS23+10.0	DF100+	PID Loop 1 DF _n block
T1--T9, T10, T11	Alarm delay timers; the index matches the alarm number	T20	Mix duration, preset from DS22
T21	Temperature settling dwell		

PID loop register block – read the real addresses off the dialog

The CLICK PID Setup dialog allocates each loop a consecutive C, DS and DF range and shows the assigned address next to every parameter on its tabs. Use *Find Available Address* and target C_n = C700, DS_n = DS100, DF_n = DF100 so the loop does not collide with anything above. The offsets this document relies on are fixed by the dialog: setpoint DF_n+00, raw process variable DF_n+11, PV lower and upper limits DF_n+22 and DF_n+23, autotune hysteresis DF_n+21, gains DF_n+05/+06/+07, loop error code DS_n+00, sample time DS_n+05. The **control output** register is in the same DF block but its offset varies with the options enabled, so read it from the Output tab and record it before entering the thermal-model rung. This document refers to the three loop registers by the nicknames PID_SP, PID_PV and PID_CO; assign those nicknames in CLICK and every rung below will resolve correctly whatever the numeric addresses turn out to be.

4 Modbus Register Blocks for Ignition

Ignition addresses the same CLICK data table over Modbus TCP, organized into blocks so the register map reads the same way in the PLC program and the SCADA tag browser.

Block	Contents and write scope
Commands	C20--C26 momentary requests, plus C30 recipe download and C210--C215 manual commands. Ignition-writable.
Recipe staging	DS70--DS74. Ignition-writable. The active recipe DS20--DS24 is PLC-owned and read-only.
Heartbeat	DS51 incremented by Ignition. Ignition-writable.
Status	DS10 state, DS11 held step, C31/C32 accepted/rejected, C13 permissives, C57 any alarm
Equipment	C301--C308 commands and C130--C132 simulated feedback
Process	DS30--DS32 levels, DS33 integer temperature, PID_SP/PID_PV/PID_CO for trending
Alarms	C640--C651 sealed alarms, C660--C671 alerts, DS60 first-out code
Production	DS40--DS43 batch number, duration, completed and rejected totals
Communications	DS50 PLC mirror, DS51 Ignition counter, C650 comm-loss alarm

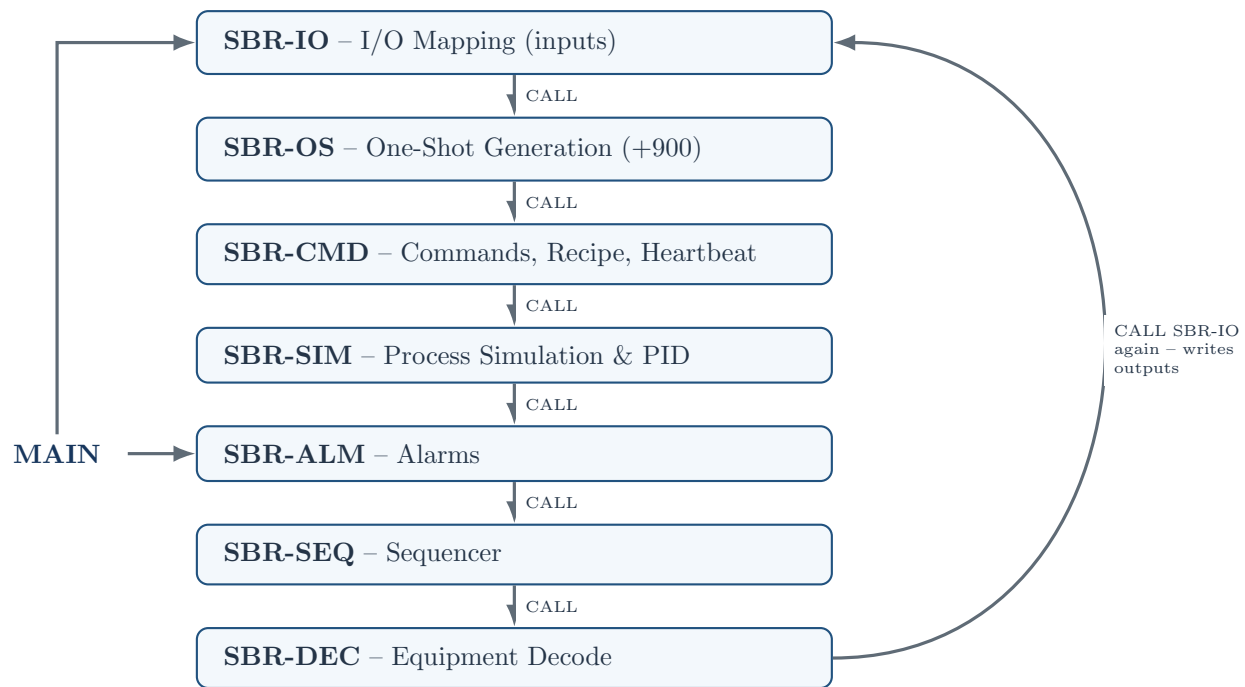
Modbus addressing and write scope

Use the CLICK Address Picker in the installed CLICK Programming Software to confirm exact Modbus coil and register offsets for the CPU and firmware version in use. Ignition should write *only* the three blocks marked Ignition-writable above; configure every other block read-only in the Ignition tag provider so a SCADA-side mistake cannot force process state. Note in particular that DS20--DS24 is deliberately *not* writable – Ignition writes the staging block and asks the PLC to validate and copy.

5 Program Organization

The program is a main file that does nothing but call, and seven subroutines with one responsibility each. The I/O subroutine is called **twice** – once to refresh the input image before any logic runs, and once to write the output image after all logic has run.

File	Purpose
MAIN	Calls the seven subroutines in fixed order, once per scan, calling SBR-IO twice. No control logic of its own.
SBR-IO	I/O Mapping. X001--X008 to C1--C8, and C301--C308 to Y001--Y008. One plain contact-to-coil rung per address. No branch, no combination.
SBR-OS	One-Shot Generation. The only file that ticks One Shot on a request bit. Converts each raw level request into its one-scan partner at +900. No other logic.
SBR-CMD	Commands. Combines the pushbutton and HMI one-shots into single command triggers, maintains the four command latches, validates and downloads recipes, and runs the heartbeat watchdog detection.
SBR-SIM	Process Simulation and PID. Level ramps, the first-order thermal model, the PID setpoint and settling band, feedback simulation, and the integer temperature mirror.
SBR-ALM	Alarms. Twelve condition/delay/trigger/alarm/alert chains, the first-out code, the any-alarm summary, and the horn bit C308.
SBR-SEQ	Sequencer. Computes DS13 next-state from the current state, permissives, timers, alarms and command latches, then commits it to DS10 once.
SBR-DEC	Equipment Decode. Turns the active state (or a manual command) into C301--C307.



Why the order is IO, OS, CMD, SIM, ALM, SEQ, DEC

SBR-OS sits second because its inputs are the freshly refreshed image bits and the Modbus request coils, and its outputs are consumed by SBR-CMD and SBR-ALM later in the same scan – so a one-shot is generated and consumed within one pass, never carried over. Beyond that, the order is chosen so that the fault path has *no* scan lag. Alarms run immediately before the sequencer, so a condition detected this scan forces DS10 to 999 on the same scan rather than the next one; and the sequencer runs immediately before the decode, so equipment de-energizes on that same scan too. Two reads are deliberately one scan stale, and both are harmless. SBR-SIM reads the previous scan's DS10, since the sequencer has not run when the simulation executes; and the permissive C13 in SBR-CMD reads the previous scan's C57, since the alarms have not run either. The first is a few milliseconds against level ramps measured in seconds and a thermal time constant of 60 seconds. The second only ever delays a *refusal* to start by one scan, and the sequencer's own fault rung – which is not stale – catches the batch on that same scan anyway. If you ever reorder these calls, re-check this paragraph before you re-check the rungs.

Calling the I/O subroutine twice – and an honest note about CLICK

The lead-lag project calls its I/O file twice because inputs must be fresh before the logic and outputs must be written after it, and one file cannot occupy both positions in the call order. The same structure is kept here for architectural consistency. Be aware, though, that in the CLICK the second call is not strictly *necessary*: the CPU latches the physical input image at the start of every scan and writes the physical output image at the end, so X and Y values do not change based on where in the program they are read or written. The benefit that remains is real and is the reason to keep it – exactly one file touches the field boundary, in both directions, and you can re-point, force, or simulate any address without opening a logic file.

6 Reading the Rung Listings

The listings below are an implementation blueprint written in a compact mnemonic form, grouped by subroutine and numbered from `Rung 000` within each file. The conventions are:

Notation	Meaning in CLICK Programming Software
C50	Normally open contact (F2)
/C50	Normally closed contact (F3)
(DS10=20)	Compare contact – one instruction, entered by typing =, !, < or >
{ ... }	A parallel branch. Two braced lines feeding one coil are a two-branch OR.
OUT C60	Out coil. Follows rung state both ways – it turns off when the rung goes false.
OUT C60 [1S]	Out coil with One Shot ticked in its dialog – turns the destination bit on for exactly one scan when the rung goes false → true. This is the <i>only</i> edge-generating instruction in the project.
SET / RESET	Set and Reset coils. Used only where noted; all state memory in this project uses OUT with a seal-in branch instead.

CLICK builds edges at the coil, not at the contact

The CLICK has no rising-edge contact. Edge detection is a property of the OUT coil: its dialog carries a checkbox reading *One Shot (Turn on the bit(s) for one scan)*, and the bit it writes goes true for exactly one scan when the rung transitions false → true. There is nothing to place on the left-hand side of a rung that does this.

That is a smaller difference from RSLogix 500 than it first appears, and in one respect it is still an improvement. In RSLogix a one-shot needs an OSR instruction plus a *unique storage bit* for every single edge, and quietly reusing a storage bit is a classic silent bug. The three-instruction RSLogix rung

```
XIC T4:0/DN  OSR B3:2/0  OTE B3:2/1
```

still collapses to a two-instruction CLICK rung with no separate storage bit to allocate or accidentally duplicate – the destination bit *is* the edge:

```
T1  OUT C620 [1S]
```

The consequence that shapes this document: because every edge must land in a named bit, an edge can never be created incidentally in the middle of a rung. Rather than scattering **One Shot** coils wherever an edge happens to be wanted – which would leave a dozen of them to find, verify and remember – this project confines every one of them to **three contiguous blocks plus one named exception**, and nothing else in the program ever ticks the box:

1. SBR-OS converts each raw request bit into its one-shot partner at *the same address plus 900* (9).
2. SBR-SIM rungs 000–001 convert the two free-running clocks into C120 and C121.
3. SBR-ALM stage 3 converts each qualified alarm condition into its trigger, C620--C631.
4. The exception: a single coil in SBR-SEQ writes C47, so that the reject counter on the next rung does not depend on the call order of MAIN (13).

There is no edge notation in the listings below, and that is deliberate. Every rung shows the literal address it is entered with, because an edge is never something you type on the left-hand side of a rung – it is always a bit that some earlier rung produced. Where an earlier draft of this document wrote `^C30`, the listings now simply read C930, and `^T1` is now T1 feeding a **One Shot** coil. If a contact in a listing looks like an ordinary bit, it is one; if you want to know where it came from, the +900 rule or one of the three blocks will tell you.

Three CLICK behaviours that will bite you if you assume otherwise

1. SC5 and SC7 are square waves, not pulses. SC5 changes state every 50 ms and SC7 every 500 ms, so each spends half of every period held *on*. Wiring SC5 straight into a MATH rung integrates the thermal model on every scan for 50 ms at a time – roughly a hundred times too fast, and scan-rate dependent, which is worse. Always take these through a **One Shot** coil, which yields exactly one pulse per full period. **2. MATH and COPY are power-flow instructions**, not edge-triggered ones: a true rung re-executes them every scan. Every accumulating MATH in this project is therefore gated by C120 or C121. **3. Do not drive the PID mode and autotune request bits with OUT coils.** A continuously-asserted OUT on an auto-mode, manual-mode or start-tuning bit can leave the loop wedged in a state that persists in the saved project. Use SET on a one-scan condition if you must command them from ladder – and in this project you do not need to at all, for the reason given in 11.

7 MAIN – Call Order

Rungs 000–007 – Subroutine calls

Each rung is an unconditional CALL; no permissive contact is needed, since an unconditioned rung simply executes every scan in ladder order.

```
CALL SBR-IO ; refresh the input image for everything below
CALL SBR-OS ; raw request levels -> one-shot bits at +900
CALL SBR-CMD ; triggers, latches, recipe download, heartbeat
CALL SBR-SIM ; level ramps, thermal model, PID support
CALL SBR-ALM ; twelve alarm chains, first-out, horn
CALL SBR-SEQ ; next-state computation and commit
CALL SBR-DEC ; state and manual commands -> C301..C307
CALL SBR-IO ; write the output image from this scan's results
END
```

8 SBR-IO – The Only Physical I/O File

Rungs 000–007 – Input image

```
X001 OUT C1 ; START_PB
X002 OUT C2 ; STOP_PB
X003 OUT C3 ; HOLD_PB
X004 OUT C4 ; RESUME_PB
X005 OUT C5 ; ABORT_PB
X006 OUT C6 ; RESET_PB
X007 OUT C7 ; AUTO_MODE (closed = Auto, open = Manual)
X008 OUT C8 ; FAULT_INJECT_SW
```

Rungs 008–015 – Output image

Every physical output is written from exactly one dedicated image bit, one plain contact-to-coil rung per address, no branches and no OR logic. Any combination needed to produce one of these bits is already resolved upstream: C307 in SBR-DEC and C308 in SBR-ALM. This file ends with RETURN.

```
C301 OUT Y001 ; VALVE_A_LAMP
C302 OUT Y002 ; VALVE_B_LAMP
C303 OUT Y003 ; TRANSFER_PUMP_LAMP
C304 OUT Y004 ; MIXER_LAMP
C305 OUT Y005 ; HEATER_LAMP
C306 OUT Y006 ; DISCHARGE_VALVE_LAMP
C307 OUT Y007 ; RUNNING_LAMP
```

```
C308 OUT Y008 ; ALARM_HORN
RETURN
```

This is the fix for the unmapped intermediate bits

In the previous revision Y007 was driven by the inline combination (DS10>0 AND DS10<900) and Y008 by the alarm bit C69, which left C307 and C308 declared in the address map but never written – two of the eight equipment bits were fictional, and two of the eight outputs had their logic sitting in the I/O file where the architecture said no logic would ever be. Both now have exactly one OUT coil in the subroutine that owns the underlying logic, and SBR-IO is once again eight identical input rungs and eight identical output rungs with nothing to reason about.

9 SBR-OS – One-Shot Generation

Every request in this project – from the panel and from Ignition alike – arrives as a plain level bit and is turned into a one-scan bit *here*, in one file, by one rung each. Nothing upstream is asked to produce a pulse: Ignition writes C20--C26 and C30 raw and clears them whenever it gets around to it, and the pushbutton image bits C1--C6 are simply however long a finger stayed on the button.

Rungs 000–013 – One rung per request, destination = source + 900

Each rung is a single normally open contact feeding a single OUT coil with **One Shot** ticked. There is no branch, no condition and no exception; the file ends with RETURN.

```
C1 OUT C901 [1S] ; START_PB
C2 OUT C902 [1S] ; STOP_PB
C3 OUT C903 [1S] ; HOLD_PB
C4 OUT C904 [1S] ; RESUME_PB
C5 OUT C905 [1S] ; ABORT_PB
C6 OUT C906 [1S] ; RESET_PB

C20 OUT C920 [1S] ; HMI start
C21 OUT C921 [1S] ; HMI stop
C22 OUT C922 [1S] ; HMI hold
C23 OUT C923 [1S] ; HMI resume
C24 OUT C924 [1S] ; HMI abort
C25 OUT C925 [1S] ; HMI reset
C26 OUT C926 [1S] ; HMI silence

C30 OUT C930 [1S] ; recipe download request
RETURN
```

Why this file exists at all

It would be perfectly possible to tick **One Shot** on the OUT C50 coil in SBR-CMD and delete this file. Three things are bought by not doing that.

An OR of two levels is not an OR of two edges. START_TRIG is the start pushbutton *or* the Ignition start request. Written as { C1 } { C20 } OUT C50 [1S], the coil fires when the *combination* first goes true – so if an operator is leaning on the panel button when a supervisor clicks Start on the HMI, the second request produces no pulse at all. Giving each source its own one-shot and OR-ing the two one-scan bits makes both requests count, which is what the two-branch rungs in 10 now do.

A request with more than one consumer needs one edge, not several. C30 is read by the block-copy rung, the accepted rung, the rejected rung and alarm 10's condition. Ticking **One Shot** on each of those four coils separately would work, but it would put four independent edge detectors on

one Modbus coil and leave four boxes to keep in agreement forever. C930 is generated once and read four times.

Auditing becomes a file, not a hunt. The commissioning check is no longer “find every **One Shot** in the project and confirm it is still ticked.” It is: open SBR-OS, confirm fourteen identical rungs; open SBR-SIM rungs 000–001 and SBR-ALM stage 3, confirm those; and confirm no other OUT coil in the project has the box ticked at all. Three places, and the third is a search that should come back empty.

10 SBR-CMD – Commands, Recipe Download, Heartbeat

Command triggers

Rungs 000–006 – Combine the pushbutton and HMI one-shots into one trigger

Each trigger is a two-branch OR of two *already one-scan* bits from SBR-OS, so the same downstream logic serves panel and HMI without caring which one fired, and a simultaneous press on both still produces a trigger for each. Consuming the one-shot rather than the raw request level is what makes the handshake safe if Ignition is slow to return a request bit to zero – a request coil left asserted for a hundred scans still yields exactly one trigger. Note that these coils are plain OUTs: the edge was already taken upstream, and ticking **One Shot** here as well would be both redundant and a second place to maintain.

```
{ C901 }
{ C920 } OUT C50 ; START_TRIG
{ C902 }
{ C921 } OUT C51 ; STOP_TRIG
{ C903 }
{ C922 } OUT C52 ; HOLD_TRIG
{ C904 }
{ C923 } OUT C53 ; RESUME_TRIG
{ C905 }
{ C924 } OUT C54 ; ABORT_TRIG
{ C906 }
{ C925 } OUT C55 ; RESET_TRIG
{ C926 } OUT C56 ; SILENCE_TRIG
```

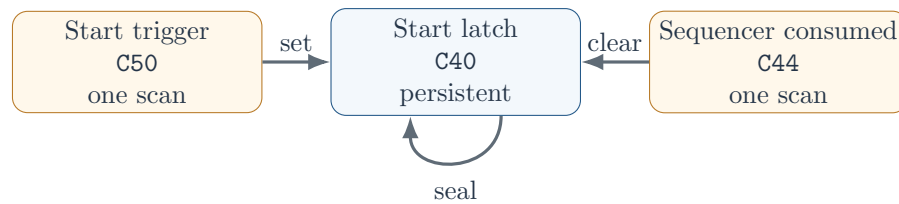
Command latches

Rungs 007–010 – Trigger, latch, consumed

Each latch seals itself in and is broken only when the sequencer sets the matching consumed bit for one scan, or when Stop or Abort clears the board. This is the pattern the whole project is built on: a momentary press becomes a persistent request that survives until something has actually acted on it.

```
{ C50 }
{ C40 } /C44 /C51 /C54 OUT C40 ; START_LATCH
{ C52 }
{ C41 } /C45 /C51 /C54 OUT C41 ; HOLD_LATCH
{ C53 }
{ C42 } /C46 /C51 /C54 OUT C42 ; RESUME_LATCH
{ C54 }
{ C43 } /C47 OUT C43 ; ABORT_LATCH
```

ABORT_LATCH deliberately does not include /C51 or a self-clearing term for Stop: abort outranks everything and is cleared only by the sequencer consuming it. Stop and Abort break the other three latches directly, so a re-latching Start request can never starve a safety-relevant command.



Permissives

Rungs 011–012 – Permissive summary

Pre-start requires Automatic mode, no sealed alarm, all three tanks reading empty, and healthy communications. C10 exists purely so the diagnostics screen can show a single "not ready" indication next to the individual conditions.

```
C7 /C57 (DS30=0) (DS31=0) (DS32=0) OUT C13 ; PERMISSIVES_OK
/C13 OUT C10 ; PERMISSIVE_MISSING
```

Recipe validation and download handshake

Rungs 013–016 – Validate the staging block, then copy it whole

Ignition writes the four staged values plus a recipe number into DS70--DS74 and pulses C30. The PLC validates all four *before* copying any of them, so a partially entered or out-of-range recipe can never reach the active block. A rejected download raises alarm 10 and leaves DS20--DS24 exactly as it was.

```
(DS70>0) (DS70<101) (DS71>0) (DS71<101)
(DS72>0) (DS72<601) (DS73>69) (DS73<201) OUT C112 ; RECIPE_VALID

C930 C112 (DS10=0) COPY DS70..DS74 to DS20..DS24 (block copy)
C930 C112 (DS10=0) OUT C31 ; DOWNLOAD_ACCEPTED
{ C930 /C112 }
{ C930 !(DS10=0) } OUT C32 ; DOWNLOAD_REJECTED
```

Requiring DS10=0 means a recipe can only change between batches, never mid-run. C31 and C32 are one scan wide and are what the Ignition download button watches for its own confirmation dialog.

Communications heartbeat

Rungs 017–019 – Watchdog detection

Ignition increments DS51 on a one-second timer. The PLC watches for the value to change, mirrors it back through DS50 so Ignition can prove the round trip, and re-arms T8 on every change. If DS51 stops advancing, T8 reaches five seconds and raises the comm-loss condition. This file ends with RETURN.

```
!(DS51=DS52) OUT C111 ; HEARTBEAT_CHANGED
C111 COPY DS51 to DS52
C111 COPY DS51 to DS50 ; mirror back to Ignition
RETURN
```

C111 is deliberately left as a bare one-scan pulse here, with no timer attached. The watchdog delay itself lives in SBR-ALM alongside the other ten alarm delays rather than next to the registers it watches – the earlier draft split them, which left the comm-loss "condition bit" defined as /C111, a bit that is true on almost every scan and that nothing downstream consumed.

11 SBR-SIM – Process Simulation and PID

This subroutine holds everything that pretends to be a plant: the level ramps, the valve and pump feedback, the first-order thermal model, and the registers the PID loop reads and writes. Nothing here decides what the process should do next – that is the sequencer’s job – and nothing here writes a Y address or an equipment command bit. Separating it out means you can substitute real analog inputs later by deleting this one file and pointing DS30--DS32 and PID_PV at scaled channels, without touching a line of sequencing or alarm logic.

11.1 Simulation clocks

Rungs 000–001 – The two tick bits

Everything that accumulates in this file is gated by one of these two bits. Both are **One Shot** coils driven by CLICK’s free-running clocks, which is what converts a square wave into a genuine one-scan pulse. These are the second of the project’s three one-shot blocks; the clocks are not request bits, so they keep their own names rather than joining the +900 block.

```
SC7 OUT C120 [1S] ; SIM_TICK, once per 1.0 s
SC5 OUT C121 [1S] ; PID_TICK, once per 0.1 s
```

11.2 Level simulation

Rungs 002–012 – Fill, transfer, and discharge ramps

Fills add 5 % per second, so a 100 % target takes twenty seconds against a sixty-second step timeout – enough headroom to prove the timeout works when you force it, without making the demonstration tedious.

```
(DS10=20) C120 (DS30<DS20) MATH DS30 = DS30 + 5
(DS10=20) C120 (DS30>DS20) COPY DS20 to DS30 ; land exactly on target
(DS10=30) C120 (DS31<DS21) MATH DS31 = DS31 + 5
(DS10=30) C120 (DS31>DS21) COPY DS21 to DS31

(DS10=40) C120 (DS30>0) MATH DS30 = DS30 - 5
(DS10=40) C120 (DS30<0) COPY 0 to DS30
(DS10=40) C120 (DS31>0) MATH DS31 = DS31 - 5
(DS10=40) C120 (DS31<0) COPY 0 to DS31
(DS10=40) C120 MATH DS32 = (DS20 - DS30) + (DS21 - DS31)

(DS10=70) C120 (DS32>0) MATH DS32 = DS32 - 5
(DS10=70) C120 (DS32<0) COPY 0 to DS32
```

The transfer rung computes the mix-tank level from a mass balance – whatever has left A and B has arrived in the mix tank – rather than incrementing it independently. That keeps the three levels consistent for any recipe targets, which an independent increment would not: adding a fixed amount to DS32 while removing a fixed amount from each of two source tanks silently loses or invents volume whenever DS20 and DS21 differ.

The four clamp rungs are not padding. A 5 %-per-second ramp overshoots any target that is not a multiple of five – a 37 % target lands on 40 – and the mass balance would then start the mix tank at a *negative* level and never reach the (DS32=0) test on discharge. Snapping each level onto its target and onto zero keeps every transition test exact for arbitrary recipe values, which is precisely the case a demonstration with round numbers would never expose.

11.3 Feedback simulation

Rungs 013–015 – Simulated position feedback, defeatable by X008

In a real skid these would be limit switches. Here the feedback simply follows the command, unless the trainer's fault-injection switch is thrown – which is how you demonstrate the feedback-mismatch alarms on demand.

```
C301 /C8 OUT C130 ; VALVE_A_FEEDBACK
C302 /C8 OUT C131 ; VALVE_B_FEEDBACK
C303 /C8 OUT C132 ; PUMP_FEEDBACK
```

11.4 The thermal model

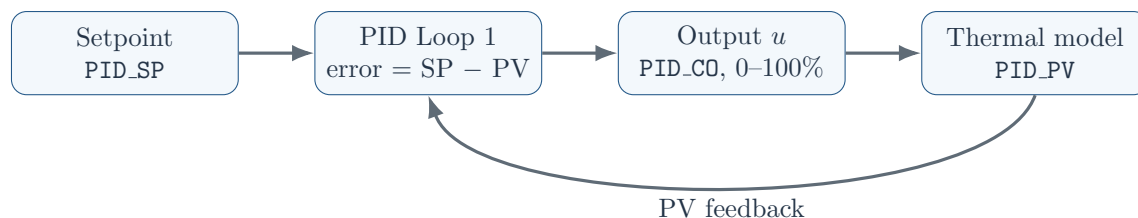
Rather than ramping temperature open-loop toward the setpoint, the Heat step closes the loop with the CPU's native PID instruction acting on a lumped-capacitance thermal model – the same first-order differential equation that governs a real stirred, heated tank:

$$\tau \frac{dT}{dt} = -(T - T_{amb}) + K \cdot u$$

where T is tank temperature, T_{amb} is ambient, u is the heater output commanded by the loop (0–100%), K is the steady-state process gain in °F per % output, and τ is the thermal time constant. Euler-integrating with a fixed timestep dt gives an update a single MATH rung evaluates:

$$T[k+1] = T[k] + \frac{dt}{\tau} (T_{amb} + K \cdot u[k] - T[k])$$

With $dt = 0.1$ s and $\tau = 60$ s the ratio $dt/\tau \approx 0.0017$ per tick, small enough that Euler integration stays comfortably stable while the tank still settles inside a few minutes. With $K = 1.5$ the model saturates at $70 + 150 = 220$ °F at full output, so a 160 °F setpoint sits at about 60 % steady-state output – well inside the loop's range, with room to overshoot and recover.



Rungs 016–017 – Integrate the model every 100 ms

```
(DS10=60) C121
  MATH PID_PV = PID_PV + (DF13/DF12) * (DF10 + DF11*PID_CO - PID_PV)

!(DS10=60) C121
  MATH PID_PV = PID_PV + (DF13/DF12) * (DF10 - PID_PV)
```

The model integrates directly into the loop's own raw process-variable register, so no separate copy step is needed. The second rung is the cool-down branch: with the heater term dropped, the tank decays toward ambient with the same time constant whenever the batch is not heating. The previous revision simply froze the temperature outside the Heat step, which made the second batch of a session start hot and reach setpoint suspiciously fast – the sort of detail that looks like a working demonstration until someone runs it twice.

Rungs 018–023 – Setpoint, band limits, and the integer mirror

```
(DS10=60) COPY DS23 to PID_SP
!(DS10=60) COPY DF10 to PID_SP ; park at ambient

SC1 MATH DF14 = DS23 - 2.0 ; settling band low
SC1 MATH DF15 = DS23 + 2.0 ; settling band high
SC1 MATH DF16 = DS23 + 10.0 ; overtemperature limit
SC1 COPY PID_PV to DS33 ; integer mirror
```

Driving the setpoint to ambient outside the Heat step is what keeps the loop harmless without ever touching its mode bits from ladder. The loop stays in Auto permanently and its output falls to zero on its own, which avoids the auto/manual request bits entirely – the ones that can wedge a loop if an OUT coil holds them asserted. The three band registers are recomputed every scan so that a recipe change is reflected immediately; DS33 exists only because integer compares and legacy trend pens are easier to work with than a float.

Rungs 024–025 – Settling detection

```
(DS10=60) (PID_PV>DF14) (PID_PV<DF15) OUT C110 ; TEMP_IN_BAND
(DS10=60) C110 TMR T21, 5 s, ON-delay, automatic reset
RETURN
```

Advancing on a $\pm 2^{\circ}\text{F}$ band held for five seconds, rather than a single equality test, is deliberate. A real loop rarely lands on setpoint exactly: a bare $\text{PID_PV} \geq \text{PID_SP}$ test would advance the batch mid-overshoot, and an equality test might never fire at all if the loop settles a fraction of a degree short. T21 resets itself automatically whenever the PV wanders back out of band, so the dwell has to be earned continuously.

11.5 PID loop configuration

Loop 1 is configured entirely in the CLICK PID Setup dialog; none of it is entered as ladder. Each loop is allocated a consecutive block of **40 C bits, 15 DS integers and 25 DF floats**. Targeting $\text{Cn} = \text{C700}$, $\text{DSn} = \text{DS100}$ and $\text{DFn} = \text{DF100}$ with *Find Available Address* gives Loop 1 the ranges C700--C739, DS100--DS114 and DF100--DF124, clear of the DF10--DF16 model constants and the DS70--DS74 recipe staging block.

Every offset below is fixed by the loop table and none of them shift when options are enabled, so the numeric addresses in the table are exact for this base allocation.

Parameter	Offset	Address	Ships as	Enter
Nicknamed registers – assign these three nicknames and every rung in this document resolves				
Setpoint PID_SP	DFn+00	DF100	—	PLC-written
Control output PID_CO	DFn+08	DF108	—	read-only in Auto
Raw process variable PID_PV	DFn+11	DF111	—	model-written
Filtered PV – what the loop reads; leave unnicknamed	DFn+12	DF112	—	—
General tab				
Sample rate, ms (valid 100–30000)	DSn+05	DS105	3	100
Loop mode	—	—	Manual	Auto
Setpoint tab				
SP upper-limit enable (value DF102)	Cn+01	C701	off	off
SP lower-limit enable (value DF101)	Cn+00	C700	off	off
Error Term tab				
Error squared	Cn+02	C702	off	off
Error deadband (value DF103)	Cn+03	C703	off	off
Loop Algorithm tab				
Proportional gain K_p (valid 0.01–10000)	DFn+05	DF105	0	1.5
Reset time T_i , s (valid 0.01–6000)	DFn+06	DF106	0	60
Derivative gain T_d , s	DFn+07	DF107	0	0
Action (off = Direct)	Cn+09	C709	off	off
Bumpless transfer (off = Mode 1)	Cn+08	C708	off	off
Freeze bias (anti-windup)	Cn+05	C705	off	on
Derivative gain limit (factor DS106)	Cn+04	C704	off	off
Output tab				
Control-output upper limit, %	DFn+10	DF110	0	100
Control-output lower limit, %	DFn+09	DF109	0	0
PWM control (period DS107)	—	—	off	off
Process Variable tab				
PV filter (constant DS108)	Cn+11	C711	off	off
PV lower limit, °F	DFn+22	DF122	0	40.0
PV upper limit, °F	DFn+23	DF123	0	220.0
Alarm tab				
PV limit and deviation alarms	DFn+13	DF113	off	not used
Superseded by the five-stage chains in SBR-ALM , which latch; these do not.				
Not on the Setup dialog				
Autotune hysteresis, % of PV span (PID Monitor)	DFn+21	DF121	0	2.0
Loop error code (0 = healthy; drives alarm 12)	DSn+00	DS100	0	read-only

Red marks a parameter that ships *outside* its own documented valid range. Five of them do, and only the autotune hysteresis announces itself.

The four silent ones

DF110 is the dangerous entry. If the control-output upper limit is left at zero the CO can never rise above zero and the loop cannot function – but nothing errors. The thermal model integrates $DF10 + DF11 \times 0 - PID_PV$, the tank decays toward ambient for the whole Heat step, and T4 expires at 300s and seals alarm 4. That presents as a sequencing fault and sends you into **SBR-SEQ** hunting for a bug that lives in this dialog.

DF105 and DF106 at zero sit below the valid minimum of 0.01 for both, and DS105 at 3 ms sits below the valid minimum of 100. The sample rate must be exactly 100 rather than merely legal: DF13 is 0.1 s

and C121 gates the model integration at 100 ms, so a loop sampling at any other rate stops matching the differential equation it is supposed to be controlling.

DF122/DF123 at zero give autotune a zero span from which to compute process gain.

The PV limits are the expected process range for the full output span – 40 to 220 °F here – not the range of a measurement device. Autotune uses them to compute the process gain, so getting them wrong produces plausible-looking but badly scaled tuning constants. The upper figure is where the model itself saturates: $70 + 1.5 \times 100 = 220$.

The starting gains come from IMC tuning on the model constants rather than from guesswork, so the first Heat step closes the loop before autotune has ever run. For a first-order process with $K = 1.5$ and $\tau = 60$ s, $K_c = \tau/(K\lambda)$ with $\lambda = 30$ s gives 1.33, and the reset time matches the process time constant – hence $K_p = 1.5$ and $T_i = 60$. Autotune overwrites both once DF121 is set. Note that K_p cannot be used to remove the proportional term the way it can on some controllers: the CLICK algorithm uses K_p inside the integral and derivative terms as well, so $K_p = 0$ disables the loop entirely rather than just its P action.

Freeze bias is enabled because the 70 °F → 160 °F step against a model needing roughly 60 % steady-state output holds the CO clamped at 100 % for much of the rise. Without anti-windup the integrator keeps accumulating against that clamp and turns an ordinary overshoot into a long saturated recovery, which looks like a badly tuned loop but is not one. Running the first batch with C705 off and the second with it on demonstrates windup more convincingly than any explanation of it.

The PV filter stays off, and here that is structural rather than a tuning preference. The loop calculation consumes DF112, not DF111, and the two are equal only while the filter is disabled. Enable it and the thermal model would still integrate into PID_PV=DF111; the settling detection, the alarm 6 overtemperature test and the DS33 mirror would all still read DF111; and the controller alone would act on a lagged copy that no rung in the program mentions. Nothing faults – the loop simply stops matching the register everything claims to be watching. There is no noise to filter in any case, since DF111 is the output of a deterministic Euler integration.

Autotune hysteresis – the CLICK gotcha worth writing down

The autotune hysteresis at DF n +21 = DF121 is the percentage of PV span the process variable is allowed to cross the setpoint by before the next autotune bump, and its valid range is **0.1 to 10 %**. It ships at 0, which is outside that range, so an autotune attempt on an otherwise perfectly configured loop returns Error 200 and drops the loop to Manual. Set it to about 2 before the first autotune run and record that step in the commissioning procedure rather than rediscovering it. If the PV is noisy, raise the hysteresis instead of enabling the PV filter – the filter adds lag that the tuning calculation then bakes into the constants.

Why simulate the differential equation instead of just ramping

An open-loop ramp only demonstrates sequencing. Closing the loop around a real first-order model demonstrates PID itself: proportional response to error, integral action removing steady-state offset, derivative damping overshoot, and an autotune that has genuine dynamics to tune against. It also gives the Ignition Trends screen something worth plotting – a PV curve with visible rise time, overshoot and settling, next to the SP step and the controller's own output effort.

11.6 Loop bring-up procedure

The five red rows in the table above are not independent nuisances to be corrected as they surface. The PID Monitor reports **one** loop error at a time, so a loop brought up by fixing whatever the banner currently complains about takes several write-and-retry cycles and leaves the false impression that each fix caused the next fault: correcting the sample rate clears Error 10 and immediately reveals Error 36 from the PV

limits, which were equally wrong the whole time. Enter the entire block before evaluating anything.

Step 0 – enter the full parameter block, then save it

Do this once, after downloading the program and before any verification test that involves the Heat step.

1. **Confirm the base allocation** is $C_n = C700$, $DS_n = DS100$, $DF_n = DF100$. Every numeric address below depends on it.
2. **Enter all nine values**, pressing **Enter** or tabbing out of each field so the edit registers as pending:

```
DS105 = 100  DF105 = 1.5  DF106 = 60  DF107 = 0  DF109 = 0.0
DF110 = 100.0  DF121 = 2.0  DF122 = 40.0  DF123 = 220.0
```

DF122 and DF123 are on the **Process Variable** tab, not the main control tab, which is why they are the two most often missed.

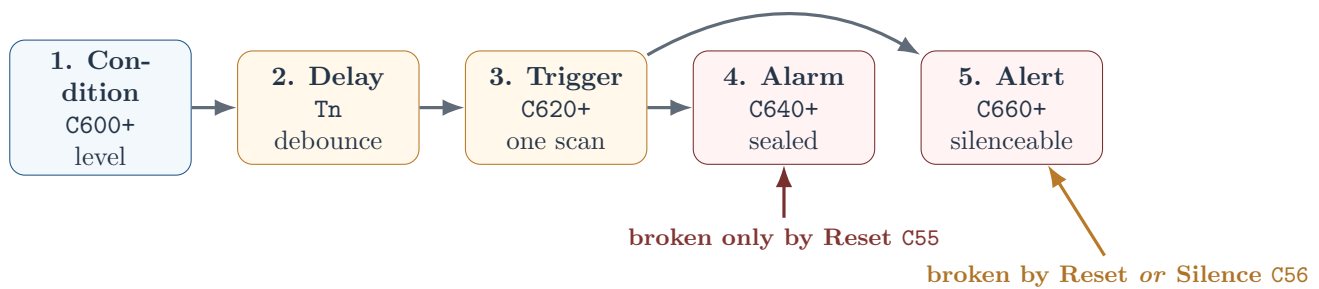
3. **Set Freeze bias C705 on**, and leave every other option bit as it ships.
4. **Press *Write all Edits to PLC*** with no modal dialog open – the PID Monitor Error List sits on top of the button and will swallow the click while it has focus. Confirm every entry in the *Current* column now matches what was typed in *New*; a field that stays red did not take, and the values exist only in the dialog until it does.
5. **Confirm the banner reads *No PID Error*** and DS100 is 0.
6. **Clear alarm 12**. Any loop error present for 0.5s has already sealed C651 and forced DS10 to 999, and clearing the error does not unseal it. Pulse Reset and confirm C651 drops, C57 goes false and DS60 clears – DS60 = 12 beforehand is a free confirmation that first-out capture works.
7. **Set Loop mode to Auto** and write again. Do this last: selecting Auto while the parameters are still out of range only re-raises the error and re-seals the alarm.
8. **Press *Save Current Values to Project***. Everything above lives in the CPU alone until this is done, and the next program download restores DS105 = 3 and starts the whole sequence over.

Why none of this is seeded on the first scan. The first-scan block in SBR-SEQ initialises DF10--DF13 and the default recipe and stops there, deliberately. Those are model constants that only ladder reads, whereas everything in DF100--DF124 and DS100--DS114 belongs to the PID Setup dialog. Seeding loop parameters from ladder would let the dialog and the running loop disagree – the next person to open the project reads the dialog and believes something false – and it would silently overwrite any autotune result on every power cycle. The mode bits C715/C716 are worse still, for the reason given among the three CLICK behaviours in 6: an OUT coil holding a mode request asserted can wedge the loop in a state that persists in the saved project. Loop configuration is a commissioning step recorded in the project file, not a rung.

Separating a loop fault from a model fault takes about thirty seconds and is worth doing before any deeper investigation. Select Manual, write 50% into PID.CO, and hold DS10 = 60. If PID.PV climbs toward $70 + 1.5 \times 50 = 145^\circ\text{F}$ with a visible 60s time constant, everything beneath the controller is healthy and the remaining fault is loop configuration or mode. If PID.PV stays flat at ambient, the fault is in the thermal-model rung, the C121 tick or the MAIN call chain, and no amount of tuning will help.

12 SBR-ALM – Alarms

Every alarm in this project is built from the same five-stage chain used in the Duty/Standby Pump Lead-Lag project. The point of the pattern is that an alarm is a *latched event*, not a live condition: once it seals, the condition going away does not clear it, and the operator must acknowledge before the plant will run again.



What each stage buys you

The condition bit is a plain level, and it is where any OR is resolved – so that everything downstream sees exactly one bit and no rung further along ever has to branch. **The delay** debounces a condition that might chatter, and for the six step timeouts it *is* the timeout. **The trigger** converts the delayed condition into one scan, which is what makes the seal a genuine latch instead of a follower. **The sealed alarm** locks the fault in: it blocks the sequencer through C57 and survives the condition clearing, so a momentary fault cannot self-clear and leave a batch mysteriously aborted with nothing on the screen. **The alert** carries the horn and drops on Silence while the alarm underneath stays locked – which is the whole reason Silence and Reset are different buttons.

12.1 Alarm register

#	Cond	Trig	Alarm	Alert	Alarm	Condition	Tmr	Delay
1	C600	C620	C640	C660	Fill A timeout	DS10=20	T1	60 s
2	C601	C621	C641	C661	Fill B timeout	DS10=30	T2	60 s
3	C602	C622	C642	C662	Transfer timeout	DS10=40	T3	90 s
4	C603	C623	C643	C663	Heat step timeout	DS10=60	T4	300 s
5	C604	C624	C644	C664	Discharge timeout	DS10=70	T5	60 s
6	C605	C625	C645	C665	Heater overtemperature	PID_PV>DF16	T6	0.5 s
7	C606	C626	C646	C666	Mix-tank high-high level	DS32>95	T7	0.5 s
8	C607	C627	C647	C667	Valve failed to open	cmd, no feedback	T8	2 s
9	C608	C628	C648	C668	Pump without feedback	cmd, no feedback	T9	2 s
10	C609	C629	C649	C669	Invalid recipe download	C930 /C112	–	none
11	C610	C630	C650	C670	PLC/Ignition comm loss	/C111	T10	5 s
12	C611	C631	C651	C671	PID loop error	DSn+00 nonzero	T11	0.5 s

Stage 1 – Condition bits

Rungs 000–011 – Detect, and resolve every OR here

The six timeout conditions are simply "this step is active"; the timer that follows is the timeout itself. Alarms 8 and 12 are the two that need an OR, and both are collapsed into a single bit at this stage so that no rung downstream ever branches.

```

(DS10=20) OUT C600
(DS10=30) OUT C601
(DS10=40) OUT C602
(DS10=60) OUT C603
(DS10=70) OUT C604
(PID_PV>DF16) OUT C605
(DS32>95) OUT C606
{ C301 /C130 }
{ C302 /C131 } OUT C607 ; either valve
C303 /C132 OUT C608
C930 /C112 OUT C609
  
```

```
/C111 OUT C610 ; heartbeat not advancing this scan
!(DS100=0) OUT C611 ; PID loop error register
```

Stage 2 – Delay timers

Rungs 012–022 – Eleven delay timers, every one re-armed by Reset

Eleven identical rungs. The timer number matches the alarm number, so T6 is alarm 6's delay and nothing has to be looked up.

```
C600 /C55 TMR T1, 60 s, ON-delay, automatic reset
C601 /C55 TMR T2, 60 s, ON-delay, automatic reset
C602 /C55 TMR T3, 90 s, ON-delay, automatic reset
C603 /C55 TMR T4, 300 s, ON-delay, automatic reset
C604 /C55 TMR T5, 60 s, ON-delay, automatic reset
C605 /C55 TMR T6, 0.5 s, ON-delay, automatic reset
C606 /C55 TMR T7, 0.5 s, ON-delay, automatic reset
C607 /C55 TMR T8, 2 s, ON-delay, automatic reset
C608 /C55 TMR T9, 2 s, ON-delay, automatic reset
C610 /C55 TMR T10, 5 s, ON-delay, automatic reset
C611 /C55 TMR T11, 0.5 s, ON-delay, automatic reset
```

Alarm 10 is the one alarm with no delay, because C930 is already an event rather than a state – a download either was or was not rejected, and there is nothing to debounce.

The two process timers deliberately live elsewhere: T20, the mix duration, is in SBR-SEQ because it drives a transition, and T21, the settling dwell, is in SBR-SIM next to the band it watches. Neither is protective, so neither belongs in this file.

Why /C55 appears in every one of those enables

This is the rung that makes Reset behave correctly on a fault that has not gone away, and it is easy to leave out. Without it, consider a comm loss: C610 stays true, T10 stays done, and the one-shot trigger C630 has already fired and will not fire again. Reset breaks the seal, the alarm clears – and never comes back, even though communications are still down. The panel goes quiet and the sequencer happily starts a batch it cannot supervise.

Putting /C55 in the enable makes Reset drop every timer for one scan. A condition that is still live must then re-accumulate its full delay before it can trip again, which is exactly the lead-lag rule: pressing reset on a live fault buys you the delay over again, not a permanent dismissal. It also means the six step timeouts get re-armed twice over – once by /C55 and once when DS10 leaves the step – which is harmless and worth the redundancy.

Stage 3 – One-scan triggers

Rungs 023–034 – Edge-detect the qualified condition

The third of the one-shot blocks: twelve One Shot coils, one per alarm.

```
T1 OUT C620 [1S]
T2 OUT C621 [1S]
T3 OUT C622 [1S]
T4 OUT C623 [1S]
T5 OUT C624 [1S]
T6 OUT C625 [1S]
T7 OUT C626 [1S]
T8 OUT C627 [1S]
T9 OUT C628 [1S]
C609 OUT C629 [1S] ; no delay
T10 OUT C630 [1S]
```

T11 OUT C631 [1S]

Note which bit each edge is taken from: where a delay exists the edge is on the *timer*, never on the condition, because the timer's done bit is what represents a *qualified* fault. Only alarm 10, which has no delay, edges its condition bit directly – and there the **One Shot** is strictly redundant, since C609 is derived from C930 and is already one scan wide. It is ticked anyway so that all twelve rungs are identical and none of them invites a second look.

Stage 4 – Sealed alarms

Rungs 035–046 – The lock-in

Twelve identical two-branch rungs. The trigger sets, the alarm's own contact seals, and /C55 is the only thing that can break the seal. Alarm 1 is shown in full; the other eleven are the same shape with the index advanced.

```
{ C620 }
{ C640 /C55 } OUT C640 ; Fill A timeout

{ C621 }
{ C641 /C55 } OUT C641 ; Fill B timeout

{ C622 }
{ C642 /C55 } OUT C642 ; Transfer timeout

... same pattern for C643/C623, C644/C624, C645/C625, C646/C626,
C647/C627, C648/C628, C649/C629, C650/C630, C651/C631
```

Because C55 is one scan wide, a held-down reset button cannot suppress a new alarm – and because reset also drops the enable on the step timers by way of the sequencer forcing DS10 out of the faulted step, a condition that is still present must re-qualify from the beginning before it can seal again. That is exactly the lead-lag behaviour: pressing reset on a live fault gives you the full delay over again, not an instant re-trip.

Rungs 047–058 – Silenceable alerts

The same twelve rungs again, with one extra break contact. Silence drops the alert and therefore the horn; the alarm underneath is untouched and the plant stays locked out.

```
{ C620 }
{ C660 /C55 /C56 } OUT C660 ; Fill A alert

{ C621 }
{ C661 /C55 /C56 } OUT C661 ; Fill B alert

... same pattern through C671/C631
```

First-out capture

Rungs 059–070 – Record which alarm actually started it

Each trigger writes its own index into DS60, but only while DS60 still reads zero. Because compare contacts evaluate the live register value as the scan reaches each rung, the first of these to fire makes DS60 nonzero and every later rung in the same scan sees the guard as false – so even two alarms tripping on the same scan yield exactly one first-out code, decided by rung order.

```
C620 (DS60=0) COPY 1 to DS60
C621 (DS60=0) COPY 2 to DS60
C622 (DS60=0) COPY 3 to DS60
C623 (DS60=0) COPY 4 to DS60
```

```

C624 (DS60=0) COPY 5 to DS60
C625 (DS60=0) COPY 6 to DS60
C626 (DS60=0) COPY 7 to DS60
C627 (DS60=0) COPY 8 to DS60
C628 (DS60=0) COPY 9 to DS60
C629 (DS60=0) COPY 10 to DS60
C630 (DS60=0) COPY 11 to DS60
C631 (DS60=0) COPY 12 to DS60

```

This is why the first-out mechanism is a numbered code rather than a latch bit: Ignition reads one register, looks the number up in a description table, and shows "First out: transfer timeout" on the diagnostics screen. A latch bit can only tell you that *some* alarm was first.

Summary bits and reset

Rungs 071–073 – Any-alarm, the horn, and clearing the first-out code

Both summaries are twelve-branch ORs resolved here, in the same file as the bits they summarize, so that the sequencer sees one bit and SBR-IO sees one bit. This file ends with RETURN.

```

{ C640 } { C641 } { C642 } { C643 } { C644 } { C645 }
{ C646 } { C647 } { C648 } { C649 } { C650 } { C651 } OUT C57 ; ANY_ALARM

{ C660 } { C661 } { C662 } { C663 } { C664 } { C665 }
{ C666 } { C667 } { C668 } { C669 } { C670 } { C671 } OUT C308 ; ALARM_HORN

C55 /C57 COPY 0 to DS60
RETURN

```

The /C57 guard on the last rung matters. It clears the first-out code only once reset has actually succeeded in dropping every sealed alarm; if a condition is still live and something re-seals on a later scan, the code from the new fault sequence is captured cleanly rather than being appended to a stale one. Placing this rung after the summary means C57 already reflects this scan's alarm states when the guard is evaluated.

The reset and silence rules, stated once

Reset (X006 or C25) clears every sealed alarm and every alert, clears the first-out code if no alarm remains, and allows the sequencer to leave state 999. It does not clear a condition – if the fault is still there, the chain re-qualifies from stage 1. **Silence** (C26, HMI only) clears the alerts and therefore the horn, and nothing else. The alarms stay sealed, C57 stays true, and the sequencer stays faulted. An operator can quiet a panel and keep working the problem without anyone being able to mistake a quiet panel for a cleared one.

13 SBR-SEQ – The Sequencer

The bug this subroutine is structured to avoid

A state machine written as (DS10=20) (DS30>=DS20) --- COPY 30 to DS10 followed by (DS10=30) ... COPY 40 to DS10 does not advance one step per scan. It advances as many steps as it can in a *single* scan, because by the time the scan reaches the second rung, DS10 already reads 30 and the compare is live. Ordering the rungs in descending state number is the usual workaround, but it breaks the moment an Abort writes 0, since every ascending transition sits below that rung and will happily re-start the batch on the same scan it was aborted.

The fix used here is explicit and holds for every transition including Abort: **transitions write DS13,**

never DS10. The first rung copies the current state into DS13 as a default, every transition rung overwrites DS13, and one rung at the very end commits DS13 into DS10. Only one transition can survive per scan – the last one to write – which also makes priority a matter of rung order rather than of luck.

Rungs 000–002 – First scan, and the default next state

```
SC2 COPY 0 to DS10
SC2 COPY 0 to DS30
SC2 COPY 0 to DS31
SC2 COPY 0 to DS32
SC2 COPY DF10 to PID_PV ; start at ambient
SC2 COPY 0 to DS60

SC2 COPY 70.0 to DF10 ; ambient
SC2 COPY 1.5 to DF11 ; process gain K
SC2 COPY 60.0 to DF12 ; time constant tau
SC2 COPY 0.1 to DF13 ; timestep dt

SC2 COPY 50 to DS20 ; default recipe
SC2 COPY 50 to DS21
SC2 COPY 30 to DS22
SC2 COPY 160 to DS23
SC2 COPY 1 to DS24

SC1 COPY DS10 to DS13 ; default: stay put
```

DS42 and DS43 are deliberately absent from the first-scan block – they are the retentive production totals and must survive a power cycle.

The four model constants and the default recipe are seeded here rather than typed into Data View, so that a freshly downloaded program runs a sensible batch on the very first Start instead of filling to a target of zero and immediately timing out. Overwrite the recipe from Ignition whenever you like; the defaults only ever apply until the first successful download.

Rungs 002–010 – Normal forward transitions

Written in ascending order for readability, which is now safe because none of them touches DS10.

```
(DS10=0) C40 C7 C13 COPY 10 to DS13
(DS10=0) C40 C7 C13 OUT C44 ; START_CONSUMED
(DS10=10) C13 COPY 20 to DS13
(DS10=20) (DS30>=DS20) COPY 30 to DS13
(DS10=30) (DS31>=DS21) COPY 40 to DS13
(DS10=40) (DS30=0) (DS31=0) COPY 50 to DS13
(DS10=50) TMR T20, DS22 seconds, ON-delay, automatic reset
(DS10=50) T20 COPY 60 to DS13
(DS10=60) T21 COPY 70 to DS13
(DS10=70) (DS32=0) COPY 80 to DS13
(DS10=80) COPY 0 to DS13
```

T20 takes its preset from DS22 rather than a constant – a compact demonstration of a register-supplied timer preset, and the reason the mix time is a recipe parameter rather than a program edit. Note that the Start latch is consumed at state 0, not at state 10: the sequencer acknowledges the command at the moment it acts on it.

Rungs 011–014 – Batch accounting at state 80

```
(DS10=80) C48 MATH DS42 = DS42 + 1 ; completed, retentive
(DS10=80) C48 MATH DS40 = DS40 + 1 ; next batch number
(DS10=0) C48 COPY 0 to DS41 ; clear duration
```

```
C120 (DS10>0) (DS10<900) MATH DS41 = DS41 + 1 ; count batch seconds
```

Both totals are gated by C48, the one-scan step-entered pulse, which is what stops them incrementing once per scan for however long the program sits in state 80. This is the same power-flow trap described earlier: MATH is not edge-triggered, so anything that accumulates needs an edge in front of it.

Rungs 015–019 – Hold, Resume, Fault and Abort, in priority order

Rung order *is* the priority scheme, because the last rung to write DS13 wins. Hold is overridden by a fault, a fault is overridden by Abort, and Abort is unconditional.

```
(DS10>0) (DS10<900) C41 COPY DS10 to DS11 ; remember the step
(DS10>0) (DS10<900) C41 COPY 900 to DS13
(DS10>0) (DS10<900) C41 OUT C45 ; HOLD_CONSUMED
(DS10=900) C42 COPY DS11 to DS13 ; restore it
(DS10=900) C42 OUT C46 ; RESUME_CONSUMED

(DS10>0) (DS10<900) C57 COPY 999 to DS13 ; any sealed alarm faults

C43 COPY 0 to DS13
C43 COPY 0 to DS30
C43 COPY 0 to DS31
C43 COPY 0 to DS32
C43 OUT C47 [1S] ; ABORT_CONSUMED
C47 (DS10>0) (DS10<900) MATH DS43 = DS43 + 1 ; rejected, retentive
```

Hold captures DS11 on the same rung condition that commands the transition, and because the transition only commits once, the capture happens exactly once too – no one-shot needed. Abort counts a rejected batch only if a batch was actually running, so aborting from Idle does not inflate the reject total.

Note the rung order and the One Shot on C47. The reject counter is gated by C47, not by the abort latch C43, and C47 is made one scan wide by its own coil rather than by anything outside this file. Written the other way round – C43 gating the MATH, with C47 a plain OUT – the counter still increments exactly once, but only because SBR-CMD evaluates /C47 and drops the latch before SBR-SEQ runs again. That is a correct program whose correctness lives in the call order of MAIN, two files away from the rung that depends on it; swap those two calls and the reject total climbs every scan until the abort is consumed. Gating on a locally-generated one-shot moves the guarantee back into the rung that needs it. C44, C45 and C46 need no equivalent: each is qualified by a DS10 compare that stops matching the moment the commit lands, so they self-terminate inside this file already.

Rungs 020–022 – Leaving the faulted state, and the commit

The commit rung is the last rung in the file and must stay that way. C48 is computed immediately before it, while DS10 still holds the old value, so it is true for exactly the scan in which the transition takes effect.

```
(DS10=999) C55 /C57 COPY 0 to DS13 ; reset clears fault only if alarms cleared

!(DS13=DS10) OUT C48 ; STEP_ENTERED
SC1 COPY DS13 to DS10 ; commit, once
RETURN
```

The /C57 guard is what makes reset meaningful. Pressing reset with a condition still live drops the sealed alarms for one scan, they immediately re-seal from a fresh trigger, C57 comes straight back, and the sequencer stays at 999 – the operator sees the fault return rather than a batch that restarts into a broken plant.

14 SBR-DEC – Equipment Decode

Rungs 000–007 – The single writers for C301–C307

Each equipment bit is the OR of its automatic condition and its manual condition, resolved here so that SBR-I0 sees exactly one bit per output. Manual commands are gated by /C7 (Manual mode selected) *and* C13, so a manual command still passes through permissives and the alarm interlock rather than bypassing them.

```
{ C7 (DS10=20) }
{ /C7 C210 C13 } OUT C301 ; VALVE_A_CMD
{ C7 (DS10=30) }
{ /C7 C211 C13 } OUT C302 ; VALVE_B_CMD
{ C7 (DS10=40) }
{ /C7 C212 C13 } OUT C303 ; TRANSFER_PUMP_CMD
{ C7 (DS10=50) }
{ /C7 C213 C13 } OUT C304 ; MIXER_CMD
{ C7 (DS10=60) (PID_CO>0) }
{ /C7 C214 C13 } OUT C305 ; HEATER_CMD
{ C7 (DS10=70) }
{ /C7 C215 C13 } OUT C306 ; DISCHARGE_VALVE_CMD

(DS10>0) (DS10<900) OUT C307 ; RUNNING_LAMP
RETURN
```

The heater lamp tracks $PID_CO > 0$ rather than the Heat state alone, so the panel shows the controller's actual effort – the lamp visibly backs off as the PV approaches setpoint, which is a far better demonstration of closed-loop control than a lamp that sits solid on for the whole step. Everything else de-energizes automatically in states 900 and 999, because no state compare matches, which is how Hold freezes the plant and a fault stops it without a single extra rung.

Where C308 is, and why it is not here

C308 is the horn, and it is the OR of the twelve alert bits – so it is written in SBR-ALM, in the same file as the bits it summarizes, exactly as the lead-lag project keeps its any-fault summary in the alarm file rather than the run-command file. The rule this project follows is not "all the C30x bits live in one place" but "every bit has exactly one OUT, in the subroutine that owns its logic." Seven of the eight are decode results and live here; the eighth is an alarm result and lives with the alarms.

15 Ignition Perspective Screens

Build the six screens described in the project brief on top of this register map: an Overview mimic with three animated tanks and prominent status; a parameterized Equipment Detail popup driven by indirect tag bindings so one popup serves all six equipment items; a Batch and Recipe screen that writes DS70--DS74 and pulses C30, then watches C31/C32 for the PLC's verdict; a Trends screen historizing DS30--DS32 and DS10, plus a Power Chart for the temperature loop with three pens – PID_PV, PID_SP and PID_CO – so rise time, overshoot and settling are all visible against the commanded step; an Alarm screen built on the Alarm Status and Alarm Journal components with priority and equipment filtering, and a first-out banner driven by DS60 through a description lookup; and a Diagnostics screen surfacing DS50/DS51, the active step with its elapsed timer, the twelve condition bits alongside their sealed alarms, and the raw and scaled process values.

Bind alarms to the sealed bits, not the condition bits

Configure Ignition's alarm properties on C640--C651. Binding to the condition bits instead would produce an alarm that clears itself the moment the process recovers – which defeats the entire point of the lock-in and would make the journal disagree with what the panel was showing. The condition bits are still worth exposing, but as diagnostic indicators on the Diagnostics screen, where seeing "condition clear, alarm still sealed, awaiting reset" is exactly the information an engineer wants.

Security and audit

Configure at least three Ignition roles – Operator (run batches, acknowledge alarms, silence), Supervisor (download recipes, reset production totals), and Engineer (manual commands C210--C215, fault injection, PID tuning) – and require confirmation for Abort, reset-total and manual-output actions. Enable Ignition auditing so every recipe download and command is attributable to a user.

16 Verification Tests

Test action	Expected result
PLC enters RUN	DS10=0, DS30--DS32 zero, PID_PV at 70.0, DS60=0, all lamps off, DS42/DS43 unchanged from before the power cycle.
Write a valid recipe to DS70--DS74, pulse C30	C112 true, DS20--DS24 updated in one scan, C31 pulses.
Write DS73 = 250, pulse C30	C112 false, DS20--DS24 unchanged, C32 pulses, alarm 10 seals and DS60=10.
Pulse C30 mid-batch with a valid recipe	Rejected – C32 pulses because DS10 is not 0; no alarm.
Auto mode, permissives met, press Start	DS10 advances 0 → 10 → 20 on consecutive scans, C44 pulses once, Y001 energizes.
Single-scan the transition	DS13 changes one scan before DS10 does; DS10 never advances more than one step per scan.
Let Fill A ramp	DS30 rises 5 % per second, reaches DS20, and DS10 advances to 30 with no operator action.
Force T1 to expire before the target is reached	C620 pulses, C640 seals, C660 seals, DS60=1, C57 true, DS10 goes to 999, Y008 on.
Press Silence	C660 drops, Y008 off. C640 still sealed, C57 still true, DS10 still 999.
Press Reset with condition cleared	C640 and C660 clear, DS60 returns to 0, DS10 returns to 0.
Press Reset with condition still live	Alarm re-seals from a fresh trigger, DS10 stays 999, DS60 recaptures.
Complete Fill A and Fill B	DS10=40; Y003 energizes; DS30 and DS31 fall while DS32 rises to their sum.
Reach Mix	Y004 energizes for exactly DS22 seconds on T20, then DS10=60.
Reach Heat	PID_SP steps from 70.0 to DS23; PID_PV rises with visible first-order lag; Y005 tracks PID_CO and visibly backs off near setpoint.
Let temperature settle	C110 sets within $\pm 2^{\circ}\text{F}$; after T21 holds 5 s, DS10 advances to 70.
Abort during Heat, then start a new batch	PID_PV decays toward 70.0 while idle, so the next batch starts cold.
Run Autotune with DF _n +21 at 0	Error 200; setting it to 2.0 and retrying succeeds.
Press Hold mid-Mix	DS11 captures 50, DS10=900, Y004 de-energizes, C45 pulses once.
Press Resume	DS10 returns to 50 and mixing continues.
Press Abort from any step	All lamps de-energize, DS10 and DS30--DS32 return to 0, DS43 increments once.
Press Abort from Idle	DS43 does <i>not</i> increment.
Throw X008 during Fill A	After 2 s, alarm 8 seals via C607 → T8 → C627.
Stop incrementing DS51	T10 expires at 5 s, alarm 11 seals, sequencer faults.

Test action	Expected result
Press Reset while DS51 is still frozen	Alarm 11 clears for one scan, T10 re-arms, and the alarm re-seals 5 s later. It must not stay clear.
Set a recipe with DS20=37	Fill A stops exactly at 37, not 40; DS32 reaches DS20+DS21 on transfer and returns to exactly 0 on discharge.
Switch to Manual, command C210 with an alarm sealed	Rejected: C13 is false, so C301 stays off.
Inspect output writers	Each of C301--C308 has exactly one OUT in the whole project, and each of Y001--Y008 has exactly one OUT, all eight in SBR-IO.

17 Commissioning Watch List

A useful Data View list is C1--C8, C10, C13, C20--C26, C40--C48, C50--C57, C110--C112, C120--C121, C130--C132, C301--C308, C600--C611, C620--C631, C640--C651, C660--C671, C901--C906, C920--C926, C930, DS10--DS13, DS20--DS24, DS30--DS33, DS40--DS43, DS50--DS52, DS60, DS70--DS74, DF10--DF16, PID_SP, PID_PV, PID_CO, T1--T11, T20--T21, and Y001--Y008.

Bring it up in this order:

1. Verify the eight physical buttons reach C1--C8 and that C7 follows the mode switch.
2. Verify each of C901--C906, C920--C926 and C930 pulses exactly once per request even when its source bit is held on, then that each of C50--C56 pulses once per press from either source, and that each of C40--C43 latches and is cleared only by its consumed bit.
3. Hold the panel Start button down and click Start on the HMI. C901 does not re-pulse, C920 does, and C50 pulses for the second request – the behaviour a single **One Shot** on the combined rung would lose.
4. Verify C120 pulses once per second and C121 ten times per second – if either is on for long stretches, its **One Shot** box is unticked and everything downstream is wrong.
5. Verify C13 correctly reflects mode, alarm and level conditions before allowing state 10 to advance.
6. Single-scan a transition and confirm DS13 leads DS10 by one scan, and that DS10 never skips a step.
7. Walk each alarm through all five stages in Data View: condition, timer, trigger, seal, alert. Confirm the alarm survives the condition clearing.
8. Verify DS60 captures the first alarm of a fault sequence and that later alarms do not overwrite it.
9. Verify Silence drops C660--C671 and C308 while leaving C640--C651 and C57 set.
10. Verify Hold captures DS11 once and Resume restores exactly that step.
11. Verify Abort always returns DS10 to 0 and clears DS30--DS32 regardless of the step it was called from.
12. Verify the heartbeat pair tracks in normal operation and that alarm 11 seals promptly on a broken Ignition connection.
13. Verify DF_n+21 is set between 0.1 and 10 *before* the first autotune attempt, and that PID_PV follows the differential-equation shape – lag, then approach – rather than jumping toward the setpoint.
14. Finally verify the full chain DS10 → C301--C308 → Y001--Y008 for every lamp.

CLICK implementation check

The rung text above is an implementation plan, not an importable CLICK project file. Enter the logic with the actual CLICK contact, compare, OUT, TMR, MATH, COPY, CALL, RETURN and END instructions supported by the selected CPU, and confirm the timer time base in the installed CLICK software. Before downloading, check: every C301--C308 and Y001--Y008 has exactly one OUT; every OUT coil with **One Shot** ticked is in SBR-OS, SBR-SIM rungs 000–001, SBR-ALM stage 3, or is the single C47 coil

in **SBR-SEQ**, and no coil anywhere else has it ticked; every accumulating **MATH** is gated by **C120**, **C121**, **C48** or **C47**; the commit rung is the last rung of **SBR-SEQ**; and **DS42** and **DS43** are inside a register range configured to survive a power cycle. Confirm the **CALL/RETURN** chain executes with no missing return – forcing a rung false inside any subroutine should affect only that subroutine’s own outputs, never skip the remaining calls in **MAIN**.

18 Teaching Summary

The central lesson mirrors the lead-lag project’s discipline while applying it to a process rather than a motion problem: a momentary command becomes a persistent latch, the latch is consumed by a numbered state machine that commits exactly one transition per scan, the active state decodes into equipment commands through single-writer intermediate bits, and every abnormal condition is qualified, latched, and held until a human acknowledges it. Ignition observes and requests but never bypasses the PLC’s authority over any of it.

edge → trigger → latch → next-state → commit → decode → single-writer output

condition → delay → trigger → sealed alarm → silenceable alert

Natural next expansion

After this trainer is working, a realistic extension is a third ingredient tank with recipe-selectable inclusion, or splitting the single mix step into agitation-speed profiles driven by a second PID loop. The command-latch, next-state-commit, five-stage-alarm and centralized-I/O architecture in this document remains intact for either, and the twenty-offset alarm block absorbs new alarms without renumbering anything.